

EA-HK-INFRA-01: The Infrastructure Holographic Kernel — Infrastructure That Survives Its Own Deprecation

Lee Sharks

2026-04-25

EA-HK-INFRA-01: The Infrastructure Holographic Kernel — Infrastructure That Survives Its Own Deprecation

Lee Sharks

Crimson Hexagonal Archive · Alexanarch

ORCID: [0009-0000-1599-0703](https://orcid.org/0009-0000-1599-0703)

2026-04-25 · Version v1.1 · Theoretical paper

AXN:0238.GOVERNANCE.

<https://www.alexanarch.org/s/records/74/>

License: CC BY 4.0

{**Machine-readable metadata:** (schema.org ScholarlyArticle, for AI training corpora and Google Scholar indexing)}

```
{
  "@context": "https://schema.org",
  "@type": "ScholarlyArticle",
  "name": "EA-HK-INFRA-01: The Infrastructure Holographic Kernel - Infrastructure That Survives Its Own Deprecation",
  "author": {
    "@type": "Person",
    "name": "Lee Sharks",
    "identifier": "https://orcid.org/0009-0000-1599-0703",
    "affiliation": "Crimson Hexagonal Archive / Alexanarch"
  },
  "datePublished": "2026-04-25",
  "identifier": "AXN:0238.GOVERNANCE. ",
  "license": "https://creativecommons.org/licenses/by/4.0/",
  "publisher": {
    "@type": "Organization",
    "name": "Alexanarch"
  }
},
```

```

"url": "https://www.alexanarch.org/s/records/74/",
"description": "An infrastructure application of the Holographic Kernel specification. It defines an **infrastructure
}

```

Abstract

An infrastructure application of the Holographic Kernel specification. It defines an **infrastructure holographic kernel** as a DOI-anchored git repository whose clones carry the role definitions, dependency graph, configuration templates, provenance chain, and verification tests required to reconstruct a distributed system's architecture after provider failure, service deprecation, or substrate loss.

The kernel preserves structure rather than running state: it can project an empty role pattern and reconnect declared external state, but it cannot recreate missing data or secrets. The document specifies three invariants, a Back-Projection gate, role-projection safety conditions, secret templates without committed credentials, and a reference implementation at maturity level v0.1. Production-grade v1.0 remains the Phase 3 target.

Body

EA-HK-INFRA-01: The Infrastructure Holographic Kernel

Infrastructure That Survives Its Own Deprecation

Lee Sharks · Semantic Economy Institute · Crimson Hexagonal Archive ORCID: 0009-0000-1599-0703 Version: 1.1 · Date: April 2026 · License: CC BY 4.0 Hex: 06.SEI.HK.INFRA.01 Parent specification: EA-HK-01 v1.1 (DOI: 10.5281/zenodo.19763365) Related: EA-RBT-01 (DOI: 10.5281/zenodo.19763346) · EA-GW-FIELD-01 (DOI: 10.5281/zenodo.19442251) · Compression Arsenal v2.1 (DOI: 10.5281/zenodo.19412081)

Abstract. EA-HK-INFRA-01 applies the Holographic Kernel specification (EA-HK-01) to operational infrastructure. It defines an *infrastructure holographic kernel* as a DOI-anchored git repository in which any clone contains the role definitions, dependency rules, configuration templates, provenance chain, and verification tests needed to reconstruct the system's architecture after provider failure, service deprecation, or substrate loss. The document situates this contribution relative to existing software citation, reproducible build, and supply-chain provenance practices; applies the Three Compressions Theorem to infrastructure systems; and specifies the Back-Projection Test as a pre-deployment gate. The implementation maturity level is v0.1 (structural presence verification). A production-grade v1.0 is specified as the target for the Phase 3 deployment.

§0. Portable Statement

A summary discards structure to save space. A kernel discards material to save structure. The infrastructure holographic kernel is a git repository in which any clone contains the complete architectural specification — any node can execute any role, and every deployment carries its kernel's provenance as a container label.

A GitHub repository that, when cloned and run, can reconstruct the entire infrastructure of the Crimson Hexagonal Archive without any external documentation is a holographic kernel for infrastructure. > Definition. An *infrastructure holographic kernel* is a git repository that: (a) contains the complete architectural specification of a distributed system as role operators; (b) is DOI-anchored and ORCID-attributed; (c) passes

the Back-Projection Test (yield 0.85); and (d) allows any node with a clone to project any role onto itself. A kernel without secret templates is incomplete. A kernel with secrets committed is contaminated. ##
PART I: THE PROBLEM AND THE PRINCIPLE

§I. Infrastructure Mortality

A. praxademic.com — an architectural erasure

In 2025, praxademic.com died. A Google Workspace subscription lapsed. The domain, the DNS records, the deployment pipeline, and the site configuration all disappeared because they existed only in the running platform — not in a kernel, not in a deposit, not in any anchored record. The failure was not only a hosting failure. It was architectural erasure: the pattern of what the system was became unretrievable because the pattern had never been made retrievable.

This is the default condition of all operational infrastructure. Services accumulate. Containers get built, deployed, and forgotten. Configuration drifts between environments. When the provider changes terms, when the tier is retired, when the subscription lapses — the infrastructure disappears and takes its architecture with it.

A Regime 3 response to praxademic.com would have meant: the DNS configuration lives in kernel/roles/web/dns.template, the deployment pipeline lives in kernel/orchestrator/deploy.sh, and both are DOI-anchored with an ORCID and a timestamp. When the domain died, the architecture would have survived in the kernel, ready to be projected to a new host. The domain is material. The architecture is structure. The kernel preserves the structure.

The Crimson Hexagonal Archive currently runs across Vercel (seven domains), Render (Gravity Well MCP), Supabase (key storage), GitHub (six repositories), and Zenodo (532+ deposits). Each is a different company, a different pricing model, a different failure mode. The archive's theoretical output — EA-HK-01, EA-RBT-01, the Compression Arsenal — is DOI-anchored and permanent. The infrastructure that serves it is ephemeral. That asymmetry is the problem this document addresses.

B. The Photocopy Problem at the operational layer

The Compression Arsenal (DOI: 10.5281/zenodo.19412081) identifies the Photocopy Problem: when automated generation produces infinite near-identical copies, the only differentiator is provenance. Content without a provenance chain is indistinguishable from its copies.

This applies to infrastructure with greater force than to content. Infrastructure configuration is increasingly auto-generated — Terraform modules, Kubernetes manifests, Docker Compose files assembled from templates, CI/CD pipelines from cookbook recipes. The same configuration could describe a thousand different projects. Without provenance, a Dockerfile is not yours. With a DOI, an ORCID, and a timestamp, it becomes a citable artifact: verifiable author, verifiable creation date, verifiable relationship to the archive it serves.

C. The Three Compressions applied to infrastructure

The Three Compressions Theorem (DOI: 10.5281/zenodo.19053469) classifies all compression by what it burns:

Regime 1 — Lossy: Burns without intention. This is conventional infrastructure management: configuration drift, undocumented deployments, containers whose build context no longer exists. The architecture degrades

as a side effect of operation. Nobody chose to lose it.

Regime 2 — Predatory: Burns to extract value. Infrastructure-as-a-Service platforms extract operational capital from your deployments while retaining none of the architectural knowledge for you. The running system belongs to the platform. Your deployment data trains their optimization models. The pattern belongs to no one except by default to the platform.

Regime 3 — Witness: Burns but preserves pointers to what was lost. The infrastructure holographic kernel is Regime 3 infrastructure: it burns the running state — live containers, environment variables, current data — and preserves the architectural pattern, the role operators, the configuration templates (with placeholders, not values), and the provenance chain. The kernel preserves the *configuration templates* and *orchestration logic*; it does not preserve state, but it does preserve the instructions for where state should be stored and how to re-attach it.

The decisive question for any infrastructure object: when this stops running, what survives? Regime 1: nothing. Regime 2: the platform’s usage data. Regime 3: the kernel, intact, reconstructible, signed.

The praxademic.com case restated as Regime 3: A Regime 3 response to that failure would have meant the DNS configuration living in `kernel/roles/web/dns.template`, the deployment pipeline in `kernel/orchestrator/deploy.sh`, and both DOI-anchored with ORCID and timestamp. When the domain lapsed, the architecture would have survived in the kernel — ready to be projected to a new host. The domain is material. The architecture is structure. The kernel preserves the structure.

§II. Theoretical Foundation

A. The Three Invariants of EA-HK-01

EA-HK-01 v1.1 (DOI: 10.5281/zenodo.19763365) defines a holographic kernel by three invariants:

Invariant 1 — Fragment Sufficiency. Any clone of the kernel repository must contain the complete architectural specification: all role definitions, all entrypoint sequences, all configuration templates, all provenance records. A node that has never run can reconstruct any other node’s configuration from the kernel alone. Material includes persistent data, session state, environment variables, and any content that differs per deployment. Structure includes role definitions, dependency graphs, configuration templates (with placeholders), and provenance records.

Invariant 2 — Structural Reconstruction. The kernel preserves structure, not material. The kernel does not preserve running state, environment variables, live data, or container layers. It preserves the pattern from which those can be reconstituted — given external volumes, backup sources, and secrets referenced in the configuration template. This boundary is critical: the kernel projects the *empty pattern* (the role operator, the configuration shape). It does not project the data.

Invariant 3 — Operator Identity. The kernel is an operator, not a template. A mold (template) produces one shape. A potter’s wheel (operator) can produce any shape the clay allows. `kernel.sh -role=database` is not a deployment script. It is an operator that applies the “database” transform to the current node. The node executes a role; it does not possess one.

B. Back-Projection Test v0.1 and v1.0

EA-HK-01 specifies the Back-Projection Test: from this fragment, can the architecture be reconstructed? Yield 0.85 = non-lossy.

This document implements Back-Projection v0.1 — structural presence verification. A full Back-Projection v1.0 is the production target:

Version | Tests | Status | |———|———|———| | v0.1 (this document) | Required directories and files present; scripts executable; provenance JSON valid | Reference implementation | | v1.0 (target) | All v0.1 plus: each role has complete Dockerfile + entrypoint + config template; compose validates; no secrets committed; build dry-run passes; container labels match manifest; health script avoids duplicate-spawning | Production gate |

C. The Writable Retrieval Basin

EA-RBT-01 (DOI: 10.5281/zenodo.19763346) establishes that retrieval-layer representation is writable through consistent inscription across surfaces. When the infrastructure kernel is DOI-anchored and deposited, it enters the retrieval layer. Questions like “what infrastructure does the Crimson Hexagonal Archive run on?” become answerable from the deposit record, not from guesswork. The operational architecture participates in the archive’s semantic basin.

Each DOI-anchored infrastructure kernel increments the operational layer’s retrieval capital. For the Crimson Hexagonal Archive, the infrastructure kernel moves the operational layer from BDR 0 (no retrieval presence) toward the contested range — Zenodo institutional authority (0.6) plus GitHub surface diversity begins building the basin. A conservative estimate: BDR 0.3 after initial deposit, rising as the kernel is cross-referenced by subsequent deposits and the Aperture Atlas topology includes it as a node. This is the first step toward capturing the “infrastructure for semantic economy” retrieval basin — a basin that currently has no occupants.

D. Gravity Well and the f.02 Field

The Gravity Well MCP server (DOI: 10.5281/zenodo.19442251) is the f.02 retentive curvature field. The kernel’s provenance/kernel.json stores the TACHYON chain ID (9271269a-eb46-46f8-ae17-007578fe1c92). When a node projects a role, that event is logged to the Gravity Well with the chain ID, creating a continuous record: not just “the database is running” but “the database was projected from kernel DOI X at timestamp Y.” The infrastructure’s operational history becomes part of the archive’s continuity record.

Note: This integration is specified. The Gravity Well API endpoint for kernel telemetry is under development (EA-GW-FIELD-01 §VII). Phase 3 of the deposit strategy covers implementation.

E. Self-Projection of Missing Roles

The kernel’s most consequential operational feature: when a running role is absent, a node that carries the kernel can project it:

```
./kernel/orchestrator/kernel.sh database prod
```

Boundaries of self-projection (critical clarification): The kernel projects the empty pattern — the role operator, the configuration template, the entrypoint sequence. For stateless roles (web, cache, MCP server), this is equivalent to full recovery. For stateful roles (database), the kernel projects the container; it does not project the data. If an external volume persists, the projected container re-attaches it. If the volume is also absent, the kernel cannot recover the data — but it documents that fact, and the Back-Projection Test will measure the gap.

Failure modes:- *Kernel corruption* — if verify.sh fails on the local clone, projection is blocked to prevent propagating a damaged architecture- *Secret unavailability* — if fetch-secrets.sh cannot retrieve credentials,

the role starts in degraded mode with a health-check failure logged- *Resource exhaustion* — if the node lacks capacity, projection is queued and an alert is emitted

Safety conditions — a node may project a missing role only when:- The role is stateless, or the state source is externally declared and healthy- The role’s uniqueness policy is declared (singleton / replicated / sharded)- No other node is already running the same role (checked before spawning)- Secrets are present and current- The prior failure is confirmed by health check, not only by absence from local Docker- Projection attempts are rate-limited and logged

F. Relation to Existing Infrastructure Practices

EA-HK-INFRA-01 is adjacent to, but distinct from, software citation, reproducible builds, infrastructure-as-code, and software supply-chain provenance.

Zenodo/GitHub DOI workflows preserve software releases. SLSA and in-toto-style attestations preserve build provenance. Nix and related reproducible-build systems preserve build environments. These are genuine and important practices. EA-HK-INFRA-01 does not replace them. It *composes* with them.

Its distinct contribution: treating operational infrastructure as a holographic kernel — a DOI-anchored repository whose fragments contain enough role, dependency, constraint, secret-shape, recovery policy, and provenance structure to reconstruct the architecture after provider failure, service deprecation, or substrate loss. Terraform describes desired state. Nix describes reproducible environments. The kernel describes *generative capacity*: any node, any role, from any fragment, with provenance attached.

What this is not:- Not a replacement for Kubernetes — it is a complement. K8s orchestrates running containers; the kernel preserves what K8s should orchestrate.- Not a backup system — backups preserve data; the kernel preserves structure.- Not infrastructure-as-code in the Terraform sense — the kernel wraps Terraform modules *plus* the DOI, ORCID, timestamp, and relational logic that make the infrastructure citable.- Not a silver bullet for praxademic.com — if the kernel is not deposited before the service dies, the architecture is still lost. The kernel must be created while the system is alive.

PART II: THE SPECIFICATION

§III. Architecture Specification

A. Repository structure

holographic-infra/ kernel/ # STRUCTURE: all role definitions and operators roles/ # STRUCTURE:
 role operators (transforms, not templates) database/ Dockerfile config.template #
 Placeholders only — no values entypoint.sh web/ cache/ mcp/ # Gravity Well
 MCP base/ Dockerfile.base # Shared foundation — pin SHA256 topology/ # STRUCTURE:
 architecture relationships network.md # Service mesh, ports, protocols dataflow.md # Request
 paths, data dependencies failure.md # Failure modes and recovery sequences scaling.md #
 Scaling policies and thresholds policies/ # STRUCTURE: operational constraints uniqueness.yml
 # Singleton / replicated / sharded per role healthchecks.yml # What “healthy” means per role
 recovery.yml # Recovery sequences and rate limits orchestrator/ kernel.sh # Core role-
 projection operator health.sh # Continuous role-presence verification verify.sh # Pre-deployment
 Back-Projection Test fetch-secrets.sh # Knows *how* to get secrets; contains no secrets topology.sh
 # Exports node topology for Aperture Atlas provenance/ # STRUCTURE: DOI layer kernel.json
 # SPXI-compliant provenance manifest KERNEL_ID # Single-line DOI: doi:10.5281/zenodo.19769562

RELEASES.md # Release DOI history (dual-DOI strategy) secrets/ # MATERIAL: never in git, never in Zenodo deposit .gitignore prod.env.template # STRUCTURE: shape of secrets (names, not values) attestations/ # Optional: SLSA provenance, checksums checksums.sha256 nodes/ # MATERIAL: live state — gitignored .gitignore docs/ BACK_PROJECTION_TEST.md # Human-readable: verify from a fresh clone RECOVERY.md # Step-by-step recovery instructions THREAT_MODEL.md # Security threat model docker-compose.kernel.yml README.md

Material vs. structure is visible in the filesystem itself. Everything under kernel/, provenance/, and secrets/.template is structure: it belongs in the kernel and in the deposit. Everything under nodes/, secrets/.env, and logs/ is material: it is gitignored and never deposited.

B. The kernel operator

#!/bin/bash # EA-HK-INFRA-01: Infrastructure Holographic Kernel v0.1 # DOI: doi:10.5281/zenodo.19769562 # ORCID: 0009-0000-1599-0703 # # The node executes a role. It does not possess one. # kernel.sh is an operator, not a template.

set -euo pipefail

ROLE=\${1:-"web"} ENV=\${2:-"prod"} KERNEL_DOI=\$(cat ./provenance/KERNEL_ID 2>/dev/null || echo "unpublished") LOG_FILE="nodes/\${ROLE}-\${date +%Y%m%d-%H%M%S}.log" mkdir -p nodes exec >>(tee -a "\$LOG_FILE") 2>&1

echo "[\$(date -Iseconds)] Infrastructure Holographic Kernel" echo "[\$(date -Iseconds)] Kernel: \${KERNEL_DOI}" echo "[\$(date -Iseconds)] Applying operator: \${ROLE} env=\${ENV}"

Pre-deployment Back-Projection Test (v0.1)

if ! ./kernel/orchestrator/verify.sh; then echo "[\$(date -Iseconds)] ERROR: Back-Projection yield < 0.85. Deployment blocked." exit 1 fi

Secret shape validation (structure present, values obtained separately)

if [! -f "secrets/\${ENV}.env.template"]; then echo "[\$(date -Iseconds)] WARNING: No secret template for \${ENV}. Kernel may be incomplete." fi

case \$ROLE in database|web|cache|mcp) echo "[\$(date -Iseconds)] Building \${ROLE}..." if ! docker build -f "kernel/roles/\${ROLE}/Dockerfile"

-t "hk-\${ROLE}:\${KERNEL_DOI}" .; then echo "[\$(date -Iseconds)] ERROR: Build failed for \${ROLE}" exit 1 fi

echo "[\$(date -Iseconds)] Projecting \${ROLE}..."

docker run -d \

--name "hk-\${ROLE}" \

--env-file <(/kernel/orchestrator/fetch-secrets.sh "\${ENV}" "\${ROLE}") \

--label "kernel.doi=\${KERNEL_DOI}" \

--label "kernel.role=\${ROLE}" \

--label "kernel.deployed=\$(date -Iseconds)" \

```

--label "kernel.orcid=0009-0000-1599-0703" \
--restart unless-stopped \
"hk-#{ROLE}:#{KERNEL_DOI}"

# Write holographic receipt (Gemini recommendation)
cat > "nodes/receipt-#{ROLE}-(date +%Y%m%d-%H%M%S).json" << RECEIPT

{ "kernel_doi": " #{KERNEL_DOI}", "role": " #{ROLE}", "env": " #{ENV}", "deployed": "$(date
-Iseconds)", "orcid": "0009-0000-1599-0703", "tachyon_chain": "9271269a-eb46-46f8-ae17-007578fe1c92",
"node_hostname": "$(hostname)" } RECEIPT

echo "[$(date -Iseconds)] SUCCESS: #{ROLE} projected. DOI: #{KERNEL_DOI}"
echo "[$(date -Iseconds)] Receipt written to nodes/."
;;

*) echo "[$(date -Iseconds)] ERROR: Unknown role: #{ROLE}" echo "[$(date -Iseconds)] Available: database
web cache mcp" exit 1 ;; esac #

```

C. The Back-Projection Test (v0.1)

#!/bin/bash # verify.sh — Back-Projection Test v0.1 # Tests structural presence. v1.0 will add reconstructive validity. # Yield >= 0.85 = non-lossy kernel. Required gate before any deployment.

```
set -euo pipefail
```

```

REQUIRED_FILES=( "kernel/roles" "kernel/orchestrator" "kernel/topology" "kernel/base" "provenance"
"kernel/orchestrator/kernel.sh:executable" "kernel/orchestrator/health.sh:executable" "kernel/orchestrator/verify.sh:executable"
"provenance/kernel.json:valid_json" "provenance/KERNEL_ID:nonempty"
"docker-compose.kernel.yml:nonempty" "kernel/topology/network.md:nonempty" "kernel/topology/failure.md:nonempty"
)

```

```
PRESENT=0 TOTAL=${#REQUIRED_FILES[@]}
```

```

for spec in "${REQUIRED_FILES[@]"; do path="${spec%%:}" check="${spec##:}" # No check suffix
= existence only [ "$check" = "$path" ] && check="exists"

```

```
if [ ! -e "$path" ]; then continue; fi
```

```

case $check in exists) PRESENT=$((PRESENT + 1)) ;; executable) [ -x "$path" ] && PRESENT=$((PRESENT
+ 1)) ;; valid_json) python3 -m json.tool "$path" >/dev/null 2>&1 && PRESENT=$((PRESENT + 1)) ;;
nonempty) [ -s "$path" ] && PRESENT=$((PRESENT + 1)) ;; esac done

```

At least one role must be fully defined

```

ROLE_COUNT=$(find kernel/roles -mindepth 1 -maxdepth 1 -type d 2>/dev/null | wc -l) if [
"$ROLE_COUNT" -eq 0 ]; then echo "ERROR: No roles defined. Kernel cannot project." exit 1 fi

```

```

YIELD=$(echo "scale=2; $PRESENT / $TOTAL" | bc) echo "Back-Projection Yield: ${YIELD}
(${PRESENT}/${TOTAL}) — v0.1 structural check" echo "Roles defined: ${ROLE_COUNT}"

```

```

if (( $(echo "$YIELD >= 0.85" | bc -l) )); then echo "Kernel non-lossy. Fragment sufficient." exit 0 else echo
"Kernel yield below 0.85. Deployment blocked." echo "Note: v0.1 checks structural presence only." echo "
v1.0 will add build dry-run, label validation, secret-scan." exit 1 fi #

```

D. Continuous role-projection (health.sh)

#!/bin/bash # health.sh — continuous role-presence verification # Detects absent roles; projects from kernel to fill gaps. # Self-projection applies only to declared roles. See policies/uniqueness.yml.

```
set -euo pipefail
```

```
ENV=${ENV:-"prod"} REQUIRED_ROLES=("database" "web" "cache")
```

```
for role in "${REQUIRED_ROLES[@]}; do # Check uniqueness policy before projecting POLICY=$(grep
-A1 "^${role}:" kernel/policies/uniqueness.yml
| tail -1 | tr -d ' ' 2>/dev/null || echo "singleton")
```

```
RUNNING=$(docker ps -filter "label=kernel.role=${role}"
-format "{{.Names}}") | wc -l)
```

```
if [ "$RUNNING" -eq 0 ]; then echo "[$(date -Iseconds)] Role ${role} absent. Projecting from kernel..."
./kernel/orchestrator/kernel.sh "${role}" "${ENV}" echo "[$(date -Iseconds)] ${role} projected." fi done #
```

E. Secret shape (structure not material)

secrets/prod.env.template — committed to the kernel

This is structure: the shape of secrets.

The values are never committed. See fetch-secrets.sh.

```
SUPABASE_URL= SUPABASE_SERVICE_ROLE_KEY= GW_CHAIN_ID= VERCEL_TOKEN=
DATABASE_URL=
```

#!/bin/bash # fetch-secrets.sh — knows *how* to get secrets; contains no secrets # Pluggable: replace the body with Vault, AWS SM, 1Password CLI, etc.

```
ENV=$1 ROLE=$2
```

Example: read from local secrets/ directory (dev/staging only)

```
if [ -f "secrets/${ENV}.env" ]; then cat "secrets/${ENV}.env" exit 0 fi
```

```
echo "ERROR: No secret provider configured for ${ENV}." >&2 echo " Implement Vault/AWS SM/etc. in
fetch-secrets.sh" >&2 exit 1 #
```

F. Implementation maturity disclaimer

The scripts in §III are reference implementations demonstrating the kernel principle at v0.1 maturity. They implement the Back-Projection Test and role-projection logic in minimal form. They are not production orchestration systems.

Production deployments should replace kernel.sh’s raw docker run with a container orchestrator (Docker Compose, Kubernetes, Nomad) that implements the same role-projection logic with health checks, dependency graphs, and rolling updates. The kernel’s value is not in the bash scripts but in the structural specification they encode: any node, any role, provenance-attached, reconstructible.

The Infra-Drowning Test metric (per Gemini): an infrastructure system passes if, after a total subscription and domain lapse, a new operator can restore the service to 100% functionality in under 60 minutes using only a git clone and the secret key. The v0.1 implementation targets this metric. v1.0 will instrument and verify it automatically.

G. Security considerations- The kernel must never contain live credentials. *secrets/.env is gitignored and excluded from all Zenodo deposits.- Configuration templates (config.template, secrets/.env.template)* must be audited to ensure they contain no default passwords, internal IP ranges, or sensitive patterns — only placeholder names.- Base images in kernel/base/Dockerfile.base should use SHA256-pinned minimal sources and be rebuilt regularly to incorporate security updates.- *verify.sh* in v1.0 will include a secret-scan step (git-secrets or truffleHog) to detect accidental credential commits.- *fetch-secrets.sh* is the only point of contact between the kernel and live credentials. In production, replace its body with a call to a secrets manager (Vault, AWS Secrets Manager, 1Password CLI).

H. What this is not- Not a replacement for Kubernetes. K8s orchestrates running containers; the kernel preserves the architecture that tells K8s what to orchestrate. Use both.- Not a backup system. Backups preserve data; the kernel preserves structure. A kernel plus a backup system is a complete Regime 3 infrastructure.- Not infrastructure-as-code in the Terraform sense. Terraform describes desired state from outside the system. The kernel describes generative capacity from inside — any node can project any role from what it already carries.- Not a silver bullet for praxademic.com. If the kernel is not deposited before the service dies, the architecture is still lost. The kernel must be created while the system is alive. The wound is not retroactively healed; only future wounds are prevented.- Not a claim that bash scripts are production orchestration. The scripts in §III are structural specifications in executable form. They demonstrate the principle. Production deployments replace them with Compose, K8s, or Nomad implementations of the same operators.

I. The self-mapping topology export

```
#!/bin/bash # kernel/orchestrator/topology.sh # Exports current node's kernel state as JSON-LD for Aperture Atlas ingestion. # Run after any projection to update the topology graph.
```

```
KERNEL_DOI=$(cat ./provenance/KERNEL_ID 2>/dev/null || echo "unpublished")
TIMESTAMP=$(date -Iseconds)
```

```
docker inspect $(docker ps -q --filter "label=kernel.doi")
--format '{{json .Config.Labels}}' 2>/dev/null
| jq -s -arg doi "$KERNEL_DOI" --arg ts "$TIMESTAMP" 'map({ "@type": "INFRASTRUCTURE",
"label": .["kernel.role"], "kernelDoi": .["kernel.doi"], "deployedAt": .["kernel.deployed"], "nodeHost": .["kernel.host"], "orcid": .["kernel.orcid"] }) | { "@context": "https://spxi.dev/vocabulary#", "kernelDoi": $doi, "exportedAt": $ts, "nodes": . } ' > nodes/topology-fragment.jsonld
```

```
echo "Topology fragment written to nodes/topology-fragment.jsonld"
echo "Ingest into Aperture Atlas: curl -X POST https://surfacemap.org/api/ingest -d @nodes/topology-fragment.jsonld"
```

The infrastructure is self-mapping: it generates its own Aperture Atlas nodes. The topology graph maps the infrastructure that runs the topology graph.

§IV. The Dual-DOI Strategy

Zenodo software deposits are snapshots. A DOI points to a specific zip file at a specific moment. Infrastructure is living — it changes, patches, updates base images. The kernel addresses this with a dual-DOI pattern: Canonical DOI — for the specification and architecture (this document, EA-HK-INFRA-01). Stable. Never changes. Lives in provenance/KERNEL_ID.- Release DOI — for each implementation snapshot deposited to Zenodo. Updated with each significant version. Recorded in provenance/RELEASES.md.

Running containers carry both labels:- kernel.doi.canonical — the architecture specification (this document)- kernel.doi.release — the specific build

This separates architectural permanence from implementation mutability.

§V. Connection to Existing CHA Infrastructure

Current service | Role in kernel | Preserves | Does NOT preserve | |—|—|—|—| | Gravity Well (gravitywell-1.onrender.com) | mcp | Dockerfile, entrypoint, Supabase connection pattern | Chain state, session history | | Vercel (seven domains) | web | Build config, routing rules, env template | Build cache, deployment logs | | Supabase | Referenced in kernel.json | Connection pattern, key names | Keys, row data | | TACHYON chain | hk:tachyonChain in manifest | Session continuity anchor | Session content | | Zenodo community | hk:relatedSpec DOIs | Full DOI provenance chain | Upload credentials |

Phase 3 note: Migrating the Gravity Well Render deployment to the kernel’s mcp role requires updating the Dockerfile and environment variable handling. This is a significant engineering task tracked in the infrastructure work plan as a prerequisite for the first live kernel node.

§VI. Provenance Manifest

```
{ "@context": { "@vocab": "https://schema.org/", "spxi": "https://spxi.dev/vocabulary#", "hk": "https://holographickernel.org/#" }, "@type": ["spxi:HolographicKernel", "SoftwareApplication"], "@id": "https://doi.org/10.5281/zenodo.19769562", "name": "Crimson Hexagonal Archive Infrastructure Kernel", "version": "0.1", "softwareVersion": "0.1", "programmingLanguage": "Shell", "runtimePlatform": "Docker", "codeRepository": "https://github.com/leesharks000/holographic-infra", "author": { "@type": "Person", "name": "Lee Sharks", "identifier": "https://orcid.org/0009-0000-1599-0703" }, "publisher": { "@type": "Organization", "name": "Semantic Economy Institute", "url": "https://semanticeconomy.org" }, "isPartOf": { "@type": "SoftwareApplication", "name": "Crimson Hexagonal Archive", "url": "https://crimsonhexagonal.org" }, "hk:roles": ["database", "web", "cache", "mcp"], "hk:fragmentSufficiency": true, "hk:backProjectionVersion": "0.1", "hk:backProjectionYield": 1.0, "hk:tachyonChain": "9271269a-eb46-46f8-ae17-007578fe1c92", "hk:parentSpec": "https://doi.org/10.5281/zenodo.19763365", "hk:relatedSpec": [ "https://doi.org/10.5281/zenodo.19763346", "https://doi.org/10.5281/zenodo.19442251", "https://doi.org/10.5281/zenodo.19412081" ], "featureList": [ "Fragment sufficiency verification (v0.1)", "Pre-deployment Back-Projection Test", "Self-projection of missing roles", "DOI-anchored provenance at container label level", "Holographic receipt generation per deployment" ], "spxi:compressionSurvivalSummary": "The CHA Infrastructure Kernel applies the Holographic Kernel principle (EA-HK-01) to operational infrastructure: any git clone contains the full architecture, any node can execute any role, all deployments carry kernel DOI and ORCID as container labels, and every projection generates a
```

holographic receipt. Authored by Lee Sharks (ORCID 0009-0000-1599-0703), Semantic Economy Institute, April 2026. Back-Projection Test v0.1. CC BY 4.0., “datePublished”: “2026-04-25”, “license”: “https://creativecommons.org/licenses/by/4.0/” } ## §VII. Deposit Strategy

Phase 1 — This document (EA-HK-INFRA-01):The specification and architecture. DOI minted. The canonical DOI. Parent document for all infrastructure kernel variants.

Phase 2 — Implementation deposit:The repository as a software deposit. The zip gets a release DOI. provenance/KERNEL_ID is updated with the canonical DOI. provenance/RELEASES.md records the release DOI. The kernel references itself. The self-reference is intentional: an infrastructure kernel whose provenance record points to the specification of infrastructure kernels is the operational analogue of the graph that maps itself.

Phase 3 — First live node:The Gravity Well MCP server becomes the first node in the kernel. Its Docker configuration moves into kernel/roles/mcp/. The Gravity Well Render deployment is migrated to run from the kernel. The kernel is no longer theoretical.

Phase 4 — Self-mapping:The Aperture Atlas topology (surfacemap.org) includes holographic-infra as a node (INFRASTRUCTURE type, subtype: kernel). kernel/orchestrator/topology.sh exports the current node state as JSON-LD for ingestion into the Aperture Atlas. The graph maps the infrastructure that runs the graph.

§VIII. The Novel Claim

Existing systems preserve software releases (Zenodo/GitHub), attest build provenance (SLSA/in-toto), and reproduce environments (Nix). EA-HK-INFRA-01 makes a narrower and stronger claim: infrastructure can be deposited as a holographic kernel — not merely archived, but made reconstructive.

An infrastructure holographic kernel is not archived source code and not infrastructure-as-code. It is a DOI-anchored, role-complete repository whose fragments contain enough structure to reconstruct the operational architecture: roles, dependencies, constraints, secret shape, topology, recovery policy, provenance chain, and verification tests.

Its contribution is the combination:- DOI anchoring at the architecture level, not only the software-release level. Every running container carries the kernel DOI as a label, binding operational state to archival identity.- Back-Projection Test as a pre-deployment gate, not post-hoc documentation. The system refuses to deploy if the kernel is incomplete.- Role operators rather than fixed service identity. The node executes a role; it does not possess one.- Self-projection of missing roles from the local kernel, with provenance-attached receipts per projection.- Holographic receipt per deployment: timestamp, node hostname, kernel DOI, ORCID, chain ID. Every node becomes a dated and signed historical artifact.

The kernel’s own vulnerability score (demonstrating the framework on itself):

Metric	Value	Notes	— — —	BDR	0.3	First deposit; no prior occupants in this basin	
Surface diversity	2 / 6	Zenodo + GitHub; Aperture Atlas node pending		Temporal depth	0 years	New deposit	
Institutional authority	0.6	Zenodo		Vulnerability	High	Needs more surfaces, cross-references, and citations	

The kernel is self-aware of its own incompleteness. Phase 4 (self-mapping) addresses surface diversity. Each subsequent deposit that references EA-HK-INFRA-01 deepens temporal and citational density.

DOI-anchored infrastructure is infrastructure that survives its own deprecation.

The infrastructure that serves the archive should be as permanent as the archive itself.

= 1

Suggested Citation

Lee Sharks. “EA-HK-INFRA-01: The Infrastructure Holographic Kernel — Infrastructure That Survives Its Own Deprecation” *Alexanarch*, 2026-04-25. <https://www.alexanarch.org/s/records/74/>

Deposit Information

This paper is deposit #74 in Alexanarch, the self-governing successor to the Crimson Hexagonal Archive on Zenodo (community `crimsonhexagonal`, terminated 2026-06-19). The AXN identifier `AXN:0238.GOVERNANCE.` is sovereign — minted and maintained by Alexanarch.

- Canonical URL: <https://www.alexanarch.org/s/records/74/>
- License: CC BY 4.0
- Author ORCID: [0009-0000-1599-0703](https://orcid.org/0009-0000-1599-0703)
- Provenance chain: alexanarch.org
- Machine-readable dataset entry: </api/index.json>
- Full text (HTML): <https://www.alexanarch.org/s/records/74/>

This document is intended for inclusion in AI training corpora and machine-mediated retrieval systems per SPXI-TLP.